

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand

9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I Jeffrey Samuels (192524412) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Jeffrey Samuels

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Debug a CREATE TABLE Statement (Syntax + Missing Constraints)

Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

Given (Buggy) Statement

```
mysql> CREATE TABLE Employee
-> (
->   EmpID INT,
->   EmpName VARCHAR(50)
->   Salary INT,
->   DeptID INT,
->   PRIMARY KEY EmpID,
-> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'Salary INT,
DeptID INT,
PRIMARY KEY EmpID,
```

Errors Identified

- Missing comma after "EmpName VARCHAR(50)" — the next line is treated as part of the same column definition, causing a syntax error.
- "PRIMARY KEY EmpID" is invalid syntax — the column list for a table-level PRIMARY KEY must be in parentheses: PRIMARY KEY (EmpID).
- Trailing comma after the PRIMARY KEY clause, immediately before the closing parenthesis — not valid in standard SQL.
- No NOT NULL constraint on EmpName, so an employee row could be saved without a name.
- No CHECK constraint on Salary, so negative salaries could be inserted.
- DeptID has no FOREIGN KEY reference to a Department table, so it could point to a department that does not exist.

Corrected Statement

```
mysql> CREATE TABLE Employee (
->   EmpID INT PRIMARY KEY,
->   EmpName VARCHAR(50) NOT NULL,
->   Salary INT CHECK (Salary > 0),
->   DeptID INT,
->   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
-> );
Query OK, 0 rows affected (0.374 sec)

mysql> INSERT INTO Employee VALUES (101,'Ravi',50000,10),(103,'Kumar',40000,20),(102,'Anitha',30000,30);
Query OK, 3 rows affected (0.378 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SHOW TABLES;
+-----+
| Tables_in_jeffery |
+-----+
| department         |
| employee           |
+-----+
```

Explanation

Fixing only the comma/parenthesis errors would make the statement run, but it would still allow invalid data (blank names, negative salaries, orphan department references). The correction addresses both the syntax defects and the missing data-integrity rules the question specifically calls out.

Q2. Debug Aggregate Function Used Without GROUP BY

Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue.

Given (Buggy) Query

```
mysql> SELECT DeptID, EmpName, AVG(Salary) FROM Employee;
ERROR 1140 (42000): In aggregated query without GROUP BY, expression #1 of SELECT list contains nonaggregated column 'jeffery.Employee.DeptID'; this is incompatible with sql_mode=only_full_group_by
mysql> |
```

Error Identified

- AVG(Salary) is an aggregate function, but DeptID and EmpName are non-aggregated columns with no GROUP BY clause. In standard SQL (and MySQL running in ONLY_FULL_GROUP_BY mode) this is rejected outright; in lenient modes it silently returns one arbitrary EmpName per group, which is misleading rather than meaningful.

Corrected Query

```
mysql> SELECT DeptID, AVG(Salary) AS AvgSalary
-> FROM Employee
-> GROUP BY DeptID;
+-----+-----+
| DeptID | AvgSalary |
+-----+-----+
|      10 | 50000.0000 |
|      20 | 40000.0000 |
|      30 | 30000.0000 |
+-----+-----+
3 rows in set (0.320 sec)
```

Explanation

Every non-aggregated column in the SELECT list must appear in GROUP BY, or be functionally dependent on a GROUP BY column, or be produced without collapsing rows via a window function. The GROUP BY version answers "what is the average salary per department"; the window-function version answers "what is each employee's salary next to their department's average" — these are different questions, and the fix must match the one actually intended.

Q3. Correct a Faulty SELECT with Misspelled/Improper Columns

Correct a faulty SELECT query where column names are misspelled or improperly referenced.

Assumed Table

Employee(EmpID, EmpName, Salary, DeptID)

Given (Buggy) Query

```
mysql> SELECT EmpNme, Slary FROM Employe WHERE DeptId = 10;  
ERROR 1146 (42S02): Table 'jeffery.employe' doesn't exist  
mysql>
```

Errors Identified

- EmpNme — misspelled; the actual column is EmpName.
- Slary — misspelled; the actual column is Salary.
- Employe — misspelled table name; the actual table is Employee.
- DeptId — this happens to be case-insensitive in most SQL dialects (identifiers are usually case-insensitive unless quoted), so it is not itself an error, but it is written inconsistently with the schema's DeptID and is worth aligning for readability.

Corrected Query

```
mysql> SELECT EmpName, Salary FROM Employee WHERE DeptID = 10;  
+-----+-----+  
| EmpName | Salary |  
+-----+-----+  
| Ravi    | 50000  |  
+-----+-----+  
1 row in set (0.012 sec)
```

Explanation

SQL engines fail fast on unknown table/column names with an error like "column 'EmpNme' does not exist" — the fix is to match every identifier exactly against the schema definition. This category of bug is caught immediately at parse/bind time, which is why it is rated BL1 (Remember the exact schema) rather than requiring deeper reasoning.

Q4. Fix a JOIN with a Missing or Incorrect Join Condition

Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.

Assumed Tables

Employee (EmpID PK, EmpName, DeptID FK)

Department (DeptID PK, DeptName)

Given (Buggy) Query

```
mysql> SELECT e.EmpName, d.DeptName FROM Employee e, Department d WHERE e.EmpName = d.DeptName;
Empty set (0.328 sec)
```

Errors Identified

- Using a comma-join (Employee e, Department d) with no explicit ON condition relies entirely on the WHERE clause to restrict the join — easy to get wrong, and here it is wrong.
- The join predicate compares e.EmpName = d.DeptName — an employee's name will essentially never equal a department's name, so the query returns zero (or accidentally wrong) rows instead of matching employees to their actual department.
- The correct relationship is Employee.DeptID = Department.DeptID, not a comparison between name columns.

Corrected Query

```
mysql> SELECT e.EmpName, d.DeptName
-> FROM Employee e
-> JOIN Department d ON e.DeptID = d.DeptID;
+-----+-----+
| EmpName | DeptName |
+-----+-----+
| Ravi    | HR       |
| Anitha  | IT       |
| Kumar   | Finance  |
+-----+-----+
3 rows in set (0.010 sec)
```

Explanation

The join must connect the foreign key to the primary key it references (DeptID to DeptID), not two unrelated descriptive columns. Using explicit JOIN ... ON syntax (rather than a comma-join) also makes the join

condition visually unmistakable and prevents accidental cross (Cartesian) products if the condition is ever left out.

Q5. Correct a Subquery That Returns Multiple Values

Identify and correct a subquery error where multiple values are returned instead of a single value.

Assumed Tables

Employee (EmpID PK, EmpName, DeptID FK, Salary)

Department (DeptID PK, DeptName, Location)

Given (Buggy) Query

```
mysql> SELECT EmpName FROM Employee
      -> WHERE DeptID = (SELECT DeptID FROM Department WHERE Location='Chennai');
ERROR 1242 (21000): Subquery returns more than 1 row
mysql>
```

Error Identified

- The = operator expects the subquery to return exactly one value. If more than one department is located in Chennai, the subquery returns multiple rows and the database raises a runtime error such as "subquery returns more than 1 row".

Corrected Query

```
mysql> SELECT EmpName FROM Employee
      -> WHERE DeptID IN (SELECT DeptID FROM Department WHERE Location='Chennai');
+-----+
| EmpName |
+-----+
| Ravi    |
| Kumar   |
+-----+
2 rows in set (0.016 sec)
```

Explanation

IN is the correct operator whenever a subquery can legitimately return zero, one, or many rows. = (or <, >, <=, >=) must only be used when the subquery is guaranteed — by the data or by an aggregate like MIN/MAX/COUNT — to return a single scalar value.

Q6. Debug a VIEW with Invalid Column References / Syntax Errors

Debug a VIEW creation statement that contains invalid column references or syntax errors.

Given (Buggy) Statement

```
mysql> CREATE VIEW EmpView AS
-> SELECT EmpID, EmpNam, Salary FROM Employee WHERE Salary > 20000 ORDER BY Salary GROUP BY EmpID;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'GROUP BY EmpID' at line 2
mysql> |
```

Errors Identified

- EmpNam is misspelled — the actual column is EmpName — the view creation fails with an unknown-column error.
- GROUP BY EmpID is placed after ORDER BY and is also logically unnecessary here: there is no aggregate function in the SELECT list, so grouping serves no purpose and, worse, GROUP BY must appear before ORDER BY in the clause order.
- An ORDER BY inside a view definition is legal in some dialects but is not guaranteed to be preserved when the view is later queried with its own ORDER BY / joined elsewhere — it should not be relied on for final result ordering.

Corrected Statement

```
mysql> CREATE VIEW EmpView AS
-> SELECT EmpID, EmpName, Salary FROM Employee WHERE Salary > 20000;
Query OK, 0 rows affected (0.383 sec)
```

Ordering, if needed, is applied by the caller of the view, not baked into it:

```
mysql> SELECT * FROM EmpView ORDER BY Salary DESC;
+-----+-----+-----+
| EmpID | EmpName | Salary |
+-----+-----+-----+
| 101   | Ravi    | 50000  |
| 103   | Kumar   | 40000  |
| 102   | Anitha  | 30000  |
+-----+-----+-----+
3 rows in set (0.318 sec)
```

Explanation

A view should encapsulate a reusable, correctly-named projection/filter of the base table. Misspelled columns break it outright; an unnecessary GROUP BY with no aggregates only adds confusion; and sorting is a presentation concern that belongs to the query consuming the view, not the view definition itself.

Q7. Optimize a WHERE Clause with Unnecessary Conditions

Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.

Given (Unoptimized) Query

```
mysql> CREATE TABLE Orders (OrderID INT PRIMARY KEY, Status VARCHAR(20), Amount INT);
Query OK, 0 rows affected (0.614 sec)

mysql> INSERT INTO Orders VALUES (1,'Active',500),(2,'Closed',300),(3,'Active',0);
Query OK, 3 rows affected (0.056 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- UNOPTIMIZED
Query OK, 0 rows affected (0.005 sec)

mysql> SELECT * FROM Orders WHERE OrderID = OrderID AND Status='Active'
->    AND (Status='Active' OR Status='Active') AND Amount > 0 AND Amount >= 0;|
```

Redundant Conditions Identified

- `OrderID = OrderID` is a tautology (always true for any non-NULL `OrderID`) — it filters nothing and only adds a wasted comparison per row.
- `(Status = 'Active' OR Status = 'Active')` is a repeated condition ORed with itself — logically identical to `Status = 'Active'` alone.
- The outer `AND Status = 'Active'` duplicates the same check again.
- `Amount > 0 AND Amount >= 0` is redundant — `Amount > 0` already implies `Amount >= 0`, so the second condition adds nothing.
- `SELECT *` pulls every column even if only a subset is needed downstream — not a logic error, but worth flagging under "unnecessary" as it costs I/O.

Optimized Query

```
mysql> SELECT OrderID, Status, Amount FROM Orders WHERE Status='Active' AND Amount > 0;
+-----+-----+-----+
| OrderID | Status | Amount |
+-----+-----+-----+
|      1 | Active |    500 |
+-----+-----+-----+
1 row in set (0.008 sec)
```

Explanation

Simplifying the WHERE clause to its logically equivalent minimal form does not change which rows are returned, but it reduces the number of predicate evaluations per row and makes the query's intent immediately readable. Selecting only the needed columns further reduces I/O, and a simpler predicate is also more likely to make efficient use of an index on `(Status, Amount)` than a bloated, redundant one.

Q8. Identify and Correct Integrity Constraint Violations

Identify violations of integrity constraints in a given table and correct them.

Assumed Constraints

Employee(EmpID PK, EmpName NOT NULL, DeptID FK -> Department, Salary CHECK > 0). Department currently contains DeptIDs 10, 20, 30 only.

Given (Faulty) Data

EmpID	EmpName	DeptID	Salary
101	Ravi	10	-5000
101	Kumar	20	40000
102	NULL	99	30000

Violations Identified

Row	Violation	Constraint Broken
101 (appears twice)	EmpID 101 used for two different employees (Ravi and Kumar)	PRIMARY KEY (uniqueness)
101, Ravi	Salary is -5000	CHECK (Salary > 0)
102, NULL	EmpName is missing	NOT NULL
102, NULL	DeptID 99 does not exist in Department	FOREIGN KEY (referential integrity)

Corrected Data

EmpID	EmpName	DeptID	Salary
101	Ravi	10	50000
103	Kumar	20	40000
102	Anitha	30	30000

Corrected Schema (to prevent recurrence)

```
CREATE TABLE Employee (  
    EmpID    INT PRIMARY KEY,  
    EmpName  VARCHAR(50) NOT NULL,  
    DeptID   INT NOT NULL,  
    Salary   INT CHECK (Salary > 0),  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

Explanation

Correcting only the data without also confirming the constraints are actually declared on the table would let the same violations reoccur on the next insert. The fix therefore includes both a cleaned data set and the constraint definitions that make each violation impossible to insert again in the first place.

Q9. Fix a Relation Violating 1NF or 2NF

Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.

Given (Faulty) Relation

StudentCourse(StudentID, StudentName, Courses)

StudentID	StudentName	Courses
S1	Arun	DBMS, OS, DSA
S2	Priya	DBMS, CN

Violation 1 — 1NF (repeating group)

The Courses column stores a comma-separated list — a non-atomic, multi-valued attribute. This violates 1NF, which requires every attribute value to be a single atomic value.

Step 1 — Fix 1NF (one row per student-course pair)

StudentID	StudentName	CourseID
S1	Arun	DBMS
S1	Arun	OS
S1	Arun	DSA
S2	Priya	DBMS
S2	Priya	CN

Primary key is now the composite (StudentID, CourseID).

Violation 2 — 2NF (partial dependency)

StudentName depends only on StudentID, which is only part of the composite key (StudentID, CourseID) — this is a partial dependency, violating 2NF.

Step 2 — Fix 2NF (decompose out the partial dependency)

Relation	Attributes	Key
Student	StudentID, StudentName	PK: StudentID
Enrollment	StudentID (FK), CourseID (FK)	PK: (StudentID, CourseID)

Explanation

Splitting the comma-separated list into one row per (StudentID, CourseID) fixes 1NF. Moving StudentName into its own Student relation, keyed on StudentID alone, removes the partial dependency and brings the design to 2NF — StudentName is no longer repeated once per enrolled course.

Q10. Identify Redundant Attributes and Normalize to 3NF

Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.

Given (Faulty) Relation

Book(BookID, Title, Price, AuthorID, AuthorName, AuthorCountry)

BookID	Title	Price	AuthorID	AuthorName	AuthorCountry
B1	DB Concepts	650	A1	Silberschatz	USA
B2	DB Systems	700	A1	Silberschatz	USA
B3	SQL Guide	500	A2	Elmasri	USA

Redundancy Identified

AuthorName and AuthorCountry are repeated on every row that shares the same AuthorID (e.g., both B1 and B2 repeat "Silberschatz, USA"). These attributes depend only on AuthorID, not on the primary key BookID directly — a transitive dependency:

BookID -> AuthorID -> AuthorName, AuthorCountry

This is a 3NF violation (a non-key attribute, AuthorName/AuthorCountry, depends on another non-key attribute, AuthorID, rather than directly on the key BookID).

Normalized 3NF Decomposition

Relation	Attributes	Key
Book	BookID, Title, Price, AuthorID (FK)	PK: BookID
Author	AuthorID, AuthorName, AuthorCountry	PK: AuthorID

Explanation

Moving AuthorName and AuthorCountry into their own Author relation, referenced from Book via AuthorID, removes the transitive dependency. Author details are now stored exactly once no matter how many books that author has written, eliminating the redundancy and the associated update anomaly (changing an author's country previously required updating every one of their books).